

Skiing with Multivariable Guide

Paige Xu 2025 Period 6

Introduction:

Skiing is a pastime that many people enjoy during the winter. What makes skiing interesting is that unlike a track runner or a car on a road, we often see skiers making their own paths on mountains. Although there is not a “correct path” down a mountain, it is often dictated by someone's capabilities and desires.

It often requires trade offs between safety and fun.

For example, the most efficient path would be the direct route of gravity, it is the fastest way down, but also the most dangerous.

On the other hand, just going across the slopes is really safe but also super slow and not efficient.

So this simulation is going to deal with this problem:

How can you use multivariable calculus to find the optimal route down a mountain?

Many concepts can be applied to mountains and skiing, like the gradient vector and directional derivatives, but what is the math behind the simulation?

At the end of this guide, I will also be explaining some common sites in skiing and making sense of them using the math we used to model the simulation.

THE BASIC IDEA

To solve this, we will create a mountain that acts as our surface. The skier is seen as a point moving through a vector field defined by our Cost Function. This Cost Function will mathematically penalize terrain that is too dangerous or too boring using directional derivatives, and gradient vector from Chapter 15

It will slowly traverse down the path by constantly taking the best solution presented in the point the skier is at, at the moment.

Part 1. The Surface

First off we need to define the mountain we will be skiing off of. This mountain can be changed to model different terrains.

We define the mountain as a function $H(x, y)$ which will give us the elevation (height) at any map coordinate (x, y) . The derivative that we want to look at is the Gradient Vector of the terrain, which we know points in the direction of steepest ascent.

$$\nabla H = \left\langle \frac{\partial H}{\partial x}, \frac{\partial H}{\partial y} \right\rangle$$

Note) Computers have built in partial derivative packages that can be downloaded, but when you use more complex mountains that are built up of many different functions, then it starts to malfunction, so I had to use the limit definition.

So to do this we would have to sample a bit on either side of the point to get a result. As a mountain is thousands of feet in elevation, it shouldn't be that big of a deal.

So in the simulation, we use

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h, y) - f(x-h, y)}{2h}$$

As a substitute for partial derivative, with $(h=1)$

But the gradient vector will always point in the direction of increasing values, but we don't ski up a hill, so then we have to reverse the direction of the vector

$$\hat{v} = \frac{-\nabla H}{|\nabla H|}$$

Anyway, this gradient vector will be the basis for our calculations.

This is what can be used to find the directional derivatives as, once you zoom in close enough to a certain point, the mountain can be approximated with a tangent plane at that point.

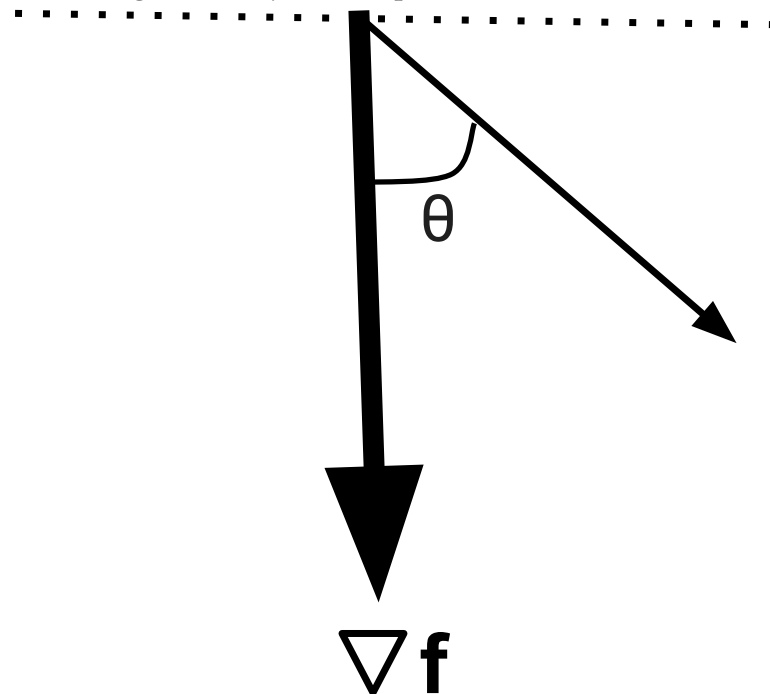
With that in mind, we want to figure out what direction to move in at each point. This means that no matter what direction we move in, it can be calculated by adjusting the angle between the gradient vector and potential direction of the skier

To illustrate that

$$\begin{aligned}
 D_u &= \nabla H \cdot \hat{u} \\
 &= |\nabla H| |\hat{u}| \cos(\theta) \quad \text{*MAGNITUDE OF UNIT VECTOR IS ONE, SO CROSS OUT} \\
 &= |\nabla H| \cos(\theta)
 \end{aligned}$$

Therefore, to find the steepness in each direction, we just have to test the angles from 0-180 degrees in front of the point.

Here is a small drawing to try and explain it. If the gradient is the steepest, then the other directional derivatives will result in a less steep slope. At each point the directional derivative will change basically the “slope” that is traveled.



This math forms the foundation of what the simulation is based on

On the mountain, areas of steeper slope are based on the magnitude of the gradient vector. To avoid these parts, we will form a cost function that will be elaborated on later.

Part 2: The Skier

The sliders on the simulation are used to map inputs to physical constraints

The **Skill** ($K \in [1, 10]$), which ranges from 1-10, will map a maximum safe slope angle for the skier of θ_{max}

FOR EXAMPLE) If $K = 1$, then the max angle may be 10°

Note) These angles are determined from data that is coming from actual mountains, thus the preferred angle will be based on angles for different colored slopes. For example, black diamond slopes are usually 40 degrees or higher.

The **Speed Preference** ($P \in [1, 10]$) will map a target velocity V_{target} for the skier

Note) The speed will largely be determined by the slope angle. A higher speed preference points to ability for steeper slopes that follow the gradient vector, whereas a low speed tolerance points to someone who would rather traverse across the slope. HOWEVER, some beginners like to take risks and go faster than their skill allows. This factor is taken into consideration when creating the path as well.

To estimate this potential speed at point (x, y) we use the slope angle as a direct proxy for potential speed

Why?) In skiing, the terminal velocity is directly correlated to the steepness of the slope, and given that other conditions like the size of each skier, or conditions are variable, we will simplify the speed to angle.

If the slope is 45 degrees, we can be pretty sure that the potential speed is high.

On the other hand, if the slope is 10 degrees, we can think of the potential speed as lower.

So you can calculate the angle with

$$Slope(u) \approx \tan^{-1}\left(\frac{f(pos+u)-f(pos)}{|u|}\right)$$

We can explain this equation bit more by saying that

1. The numerator $f(pos + u) - f(pos)$ is representative of the drop/run of the slope
2. The denominator is the length of the run
3. Taking the arctan will give you an angle of the drop, which we take as the slope angle

All that being said, the KEY difference in my simulation between these two factors is that

1. Skill is a “Hard Constraint”

This means that the skill level determines the maximum steepness that you probably would feel comfortable doing. It creates a cutoff for what steepness you do, no matter if you like speed or not.

So, when the algorithm sees a potential path, it will check if the angle is greater than your skill level allows. If it is, then it will delete it from the list of options, you will crash if you go that way.

2. Speed is a target

Once you are left with a safe path, the speed preference will decide which of those paths are the best one for you.

Here is where the cost function comes into play.

Let's define

$$C(\theta) = |\theta_{actual} - \theta_{target}|$$

This is our cost function. We want to minimize this so that it will match the user's preference.

So it will subtract the actual angle calculated with the preferred angle defined by the slider.

In conclusion, the skill function will still override the speed function because a beginner has no chance of going fast on a slope that is too steep for their skill.

That is the basic math and logic behind the calculations behind one point. As you can see, it heavily relies on the idea of gradient vectors, tangent plane approximations, and directional derivatives.

Now putting it together)

This simulation is going to implement a greedy algorithm where the decision will be based on the point traveled at the beginning. It will not look in advance all the other decisions that will happen in the future, it will travel one point at a time.

A summary of the simulation will be like this

1. Initialize a skier on top of the mountain
2. Calculate the gradient to find the Fall Line
3. Then test all the angles between the front half of the skier
4. Filter out slopes that exceed the Skill Constraint
5. Select the slope closest to the Speed Preference to travel in
6. Move the skier forward by a step distance
7. Repeat over and over until the elevation is around zero

Simulation

My completed simulation was written in python. For the visualization aspect, I imported a few python packages to deal with the GUI.

```
main.py ×  
1 import numpy as np  
2 import matplotlib  
3 matplotlib.use('TkAgg')  
4 import matplotlib.pyplot as plt  
5 from matplotlib.widgets import Slider, RadioButtons  
6 from mpl_toolkits.mplot3d import Axes3D  
7  
8
```

The simulation has a few different components: the sliders, and the mountain options. You can change between the three mountains and the sliders and the simulation will continuously compound and change.

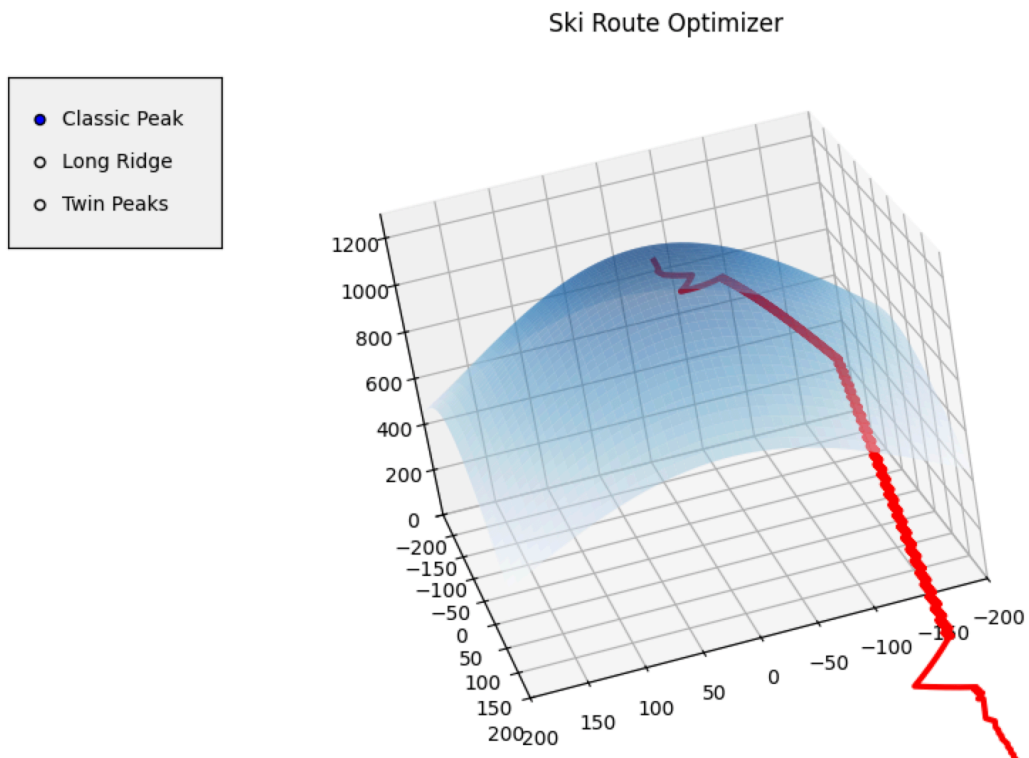
```
91 # gui widgets
92
93 # 1. mountain selector things
94 # Located at Top-Left
95 ax_radio = plt.axes([0.05, 0.7, 0.15, 0.15], facecolor='#f0f0f0')
96 radio = RadioButtons(ax_radio, list(MOUNTAIN_OPTIONS.keys()))
97
98 def change_mountain(label): 1 usage
99     global current_mountain_func
100     current_mountain_func = MOUNTAIN_OPTIONS[label]
101     update(None)
102
103 radio.on_clicked(change_mountain)
104
105 # 2. sliders
106
107 ax_skill = plt.axes([0.2, 0.1, 0.6, 0.03])
108 ax_speed = plt.axes([0.2, 0.05, 0.6, 0.03])
109
110 slider_skill = Slider(ax_skill, label: 'Skill (1-10)', valmin: 1, valmax: 10, valinit=5, valstep=1)
111 slider_speed = Slider(ax_speed, label: 'Speed (1-10)', valmin: 1, valmax: 10, valinit=5, valstep=1)
```



Skill (1-10) 5

Speed (1-10) 5

Here are some frames from the sim, and a video can be found on the last slide of my slideshow. [Slideshow](#)

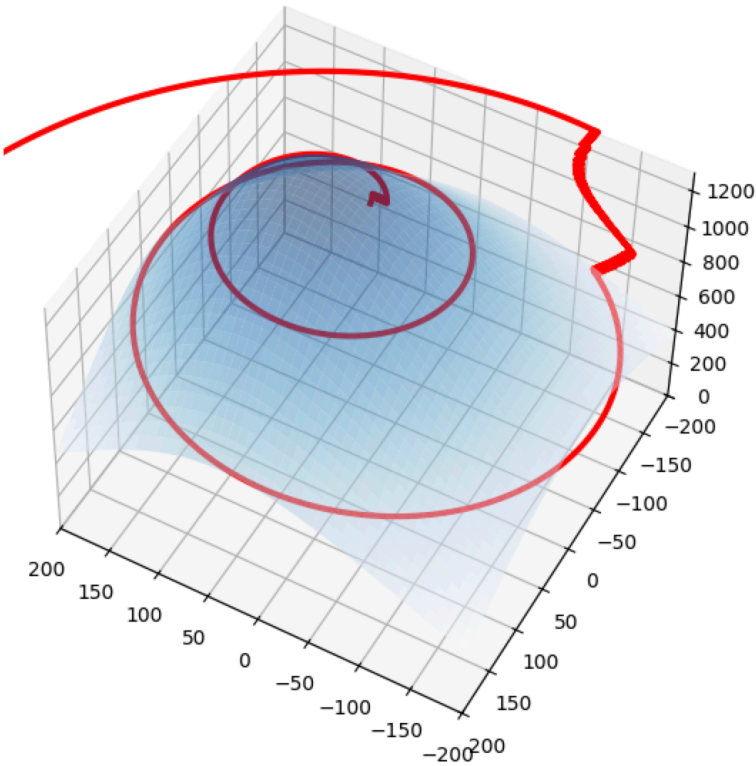


Skill (1-10) 6

Speed (1-10) 8

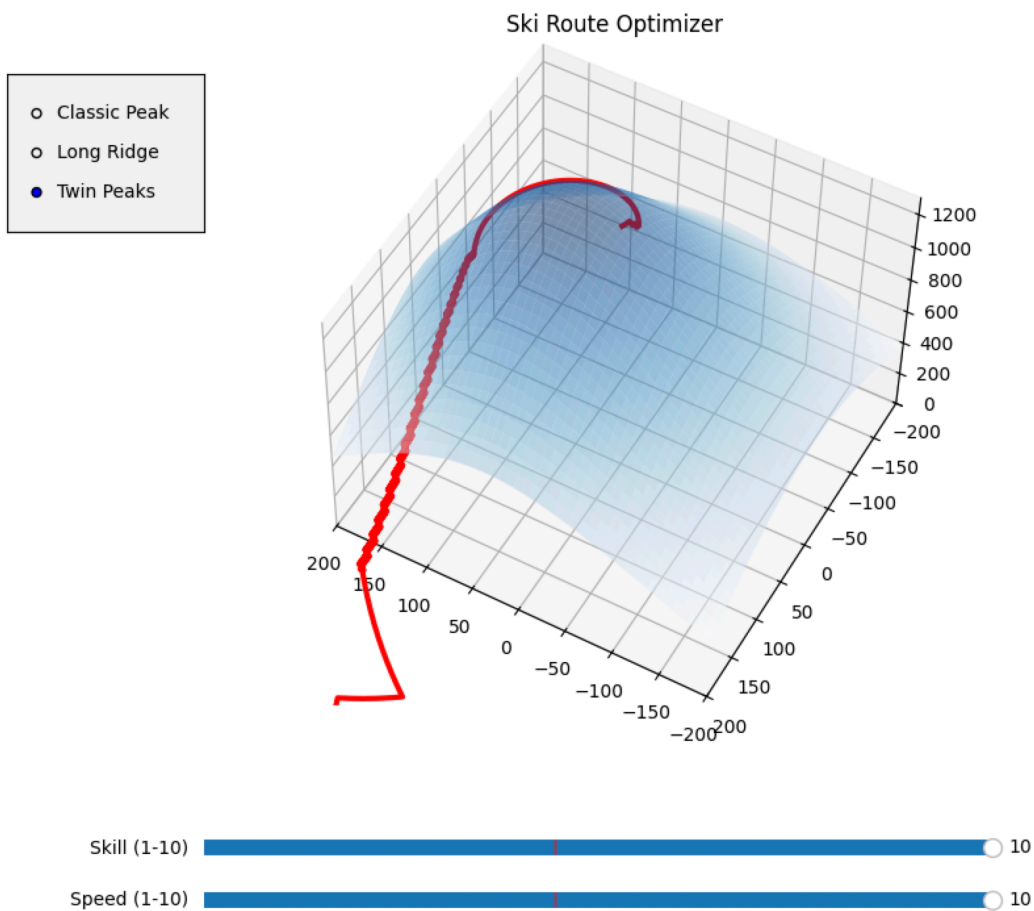
Ski Route Optimizer

- Classic Peak
- Long Ridge
- Twin Peaks

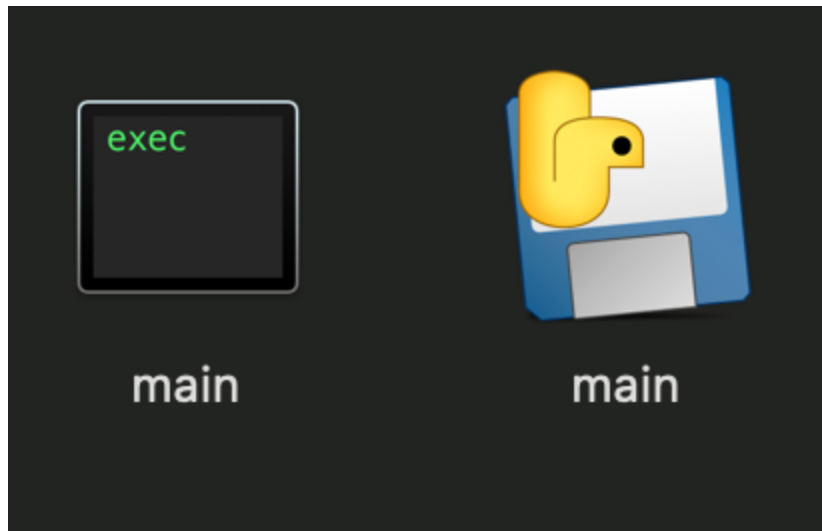


Skill (1-10) 1

Speed (1-10) 1



To run this simulation, I turned the original python file into a .exe so you directly interact with the code. When you unzip the file, there should be two packages, one is runnable if your computer is operated with Windows and the other is for Mac.



↑ For windows

↑ For Mac

To access the actual code, I have uploaded a pdf of my code as supplemental material.

You can find that here:

https://drive.google.com/file/d/1YytaXNa1HcIanFC8Yc6vjE6x_f3nsSoK/view

Broad Ideas and Connections

Obviously this simulation will negate many factors that might make this “optimal” route impossible for you to go out and attempt. However, we still can extrapolate some interesting and important ideas from this simulation that do actually hold up in real life.

You may have wondered why skiers often ski in S shapes, and now we can explain it well. When skiing in an S shape, there are only small segments when the skier is actually pointing downhill and spending more time perpendicular to the fall line.

So, instead of finishing with turning down the hill, you actually end up leaning into the face of the mountain and descending less while traversing more horizontally.

Below are example images to show you what I am talking about

So as we see here, this guy is turning down the path in a way that still traverses across the mountain more than he goes down. This large S shape will help the skier control their speed, especially when going down a steep hill.

This makes sense when we examine the directional derivative and gradient. Because he isn't travelling in the same direction as the gradient, he doesn't meet the steepest route.



Now we will see another example to illustrate the opposite solution below.

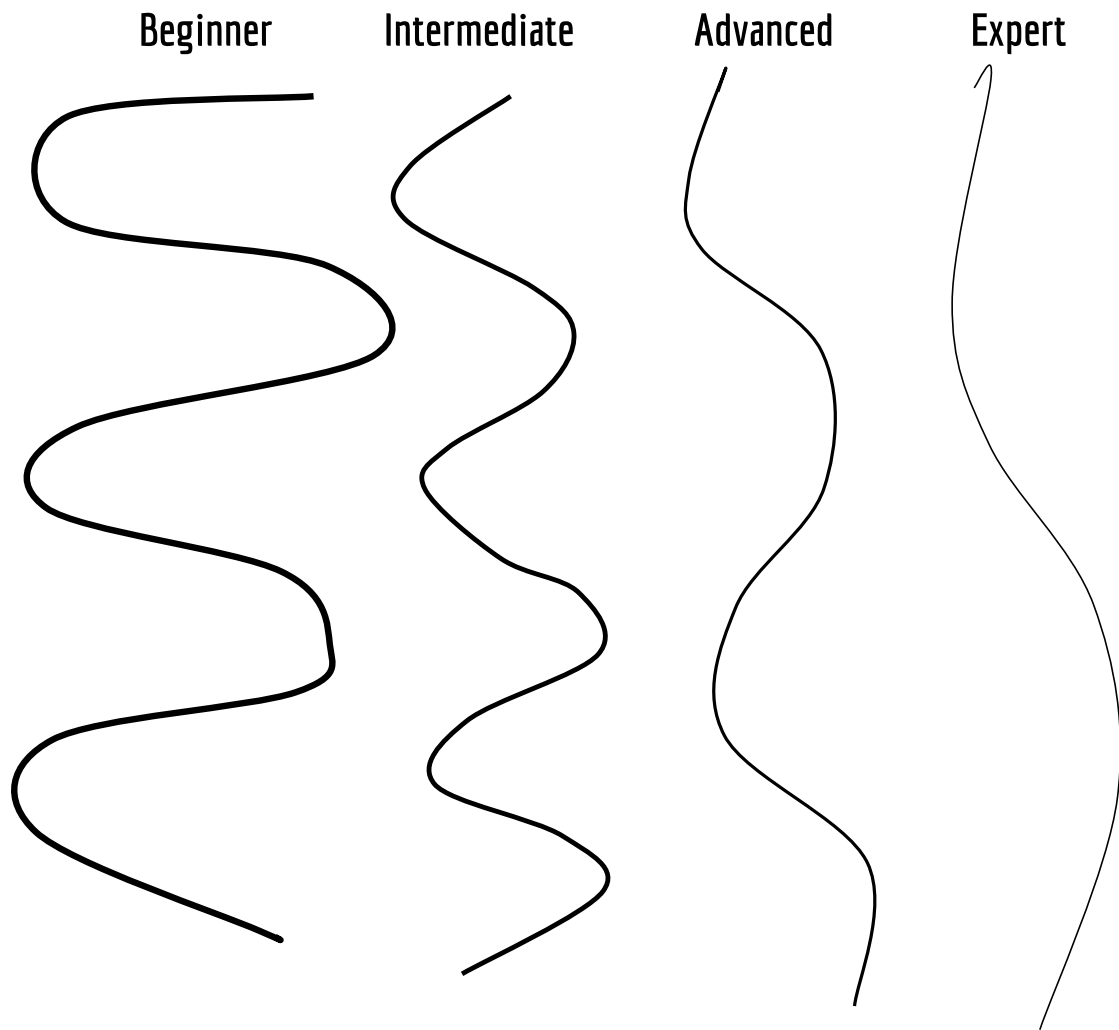
In this situation, we can see that the S shape no longer looks very S like, and has become stretched out to be almost vertical.



This will follow the gradient vector more, resulting in more speed and danger. These are experts and thus they can control their motion better and handle the speed better. This represents the tradeoff between speed and safety. As they can handle more speed, they will carve out a path that is straighter downward with less horizontal run.

In conclusion I will draw a diagram that illustrates the path that beginners usually take, up to what experts usually do to traverse a path.

Although this isn't the exact path that people of different ski levels take, this is the general pattern involving skiers.



As we see, the beginner takes large sloping S curves to maintain their balance and safety, traveling mainly horizontally while descending slowly.

Then as they become more advanced, the S shape becomes more subtle, and the frequency of turns that a skier makes can be smaller and more frequent.

Finally, the most advanced skiers can basically ski according to the gradient vector, with small segments of horizontal travel.

Reflection

Overall, this project was really interesting and I stretched my thinking about how math pervades into everyday life. When I first thought of skiing, I didn't realize how much math could actually be applied to this topic, and even now I have only scratched the surface. I would say that the most challenging part of this project was on how to approach the algorithm. Although I knew the gradient vector would play a role in the path, the rest of the math was quite murky. However, in the end, I think the idea behind the simulation is not bad as it does create differing paths that hold up to questioning. For example, if the speed is high, then the path will look more straightforward, which by reasoning we know is true. I would say that I am quite proud of this model as my programming knowledge is still limited. Utilizing the packages was a good idea as many of the widgets were easy to implement, and many of the visualizations (like the axis) were easy to learn. That being said, I think that this project could use a lot of improvement. Firstly, this algorithm is still not ideal, with the greedy algorithm presenting a bad approach. In the future, adding something that scans the mountain in advance would probably give you a closer "optimal" path. Secondly, as the mountain is quite large, the path from far away isn't very clear. You can't really see how the curve will go back and forth within the path. As a whole, I think I did a pretty good job with the product, and I had to learn a lot about coding, algorithms, and math to complete it. This project was very enjoyable and fun to complete as we had full control of our topic. This was really beneficial for me to see math in a new light, and I think I have developed a deeper appreciation and understanding of math in everyday life.

Conclusion

Skiing is a very versatile sport that has been proven to also be tied to math. Although I didn't explore how every aspect of skiing could be explained by math, the most obvious one, has been broken down by this simulation and explanation.

There is a lot more to explore, from carving to curvature, but just by examining the steepness, we can already generate a pretty good path that makes sense mathematically.

Sources

There weren't that many articles or media I found that correlated directly with this topic, but some of the information I used to create my simulation are from below sources

Sienkiewicz, Taylor. *Green, Blue and Black: How Ski Areas Rate Their Terrain*. Summit Daily, 8 Jan. 2021,

www.summitdaily.com/explore-summit/explore-summit-magazine/green-blue-and-black-how-ski-areas-rate-their-terrain/

“Setup Matplotlib on PyCharm.” *GeeksforGeeks*,

www.geeksforgeeks.org/python/setup-matplotlib-on-pycharm/

YouTube video “Geometric Meaning of the Gradient Vector” (QQPz3eXXgQI)

Geometric Meaning of the Gradient Vector, YouTube, uploaded by *Khan Academy*, n.d.,

www.youtube.com/watch?v=QQPz3eXXgQI

“What Separates a Double Black Diamond from a Black Diamond?” *SnowBrains*,

snowbrains.com/what-separates-a-double-black-diamond-from-a-black-diamond/.

“*numpy.exp* — NumPy v2.5.dev0 Manual.” *NumPy*,

numpy.org/devdocs/reference/generated/numpy.exp.html.

YouTube video “How to Convert Any Python File to .exe Using PyCharm” (BGZGt1Rqof0)

How to Convert Any Python File to .exe Using PyCharm, YouTube, uploaded by *Tech with Tim* n.d., www.youtube.com/watch?v=BGZGt1Rqof0.

Jenny Wiegand, *How to Become a Smart, Tactical Skier in Any Terrain*. *SKI*, 6 Mar. 2021,

www.skimag.com/performance/instruction/skiing-tactics-turn-shape/